```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
```

```
df=pd.read_csv('diabetes.csv')
```

```
df
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1
...	...	...	...	...	...	...	...	...	...
763	10	101	76	48	180	32.9	0.171	63	0
764	2	122	70	27	0	36.8	0.340	27	0
765	5	121	72	23	112	26.2	0.245	30	0
766	1	126	60	0	0	30.1	0.349	47	1
767	1	93	70	31	0	30.4	0.315	23	0

768 rows × 9 columns

Next steps: Generate code with df New interactive sheet

```
df.isnull().sum()
```

	0
Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
df.describe()
```

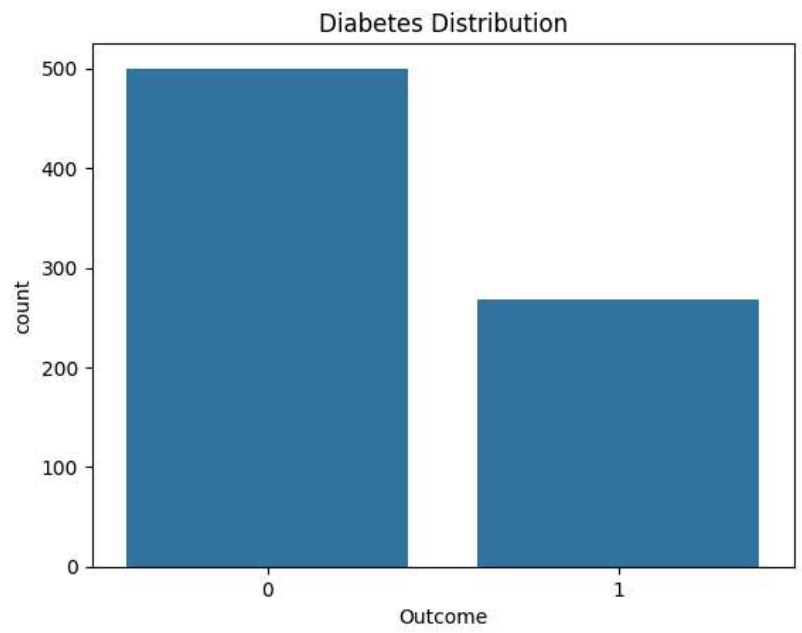
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	0.471876	33.240885	0.348958
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	0.331329	11.760232	0.476951
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.078000	21.000000	0.000000
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

```
df['Outcome'].value_counts()
```

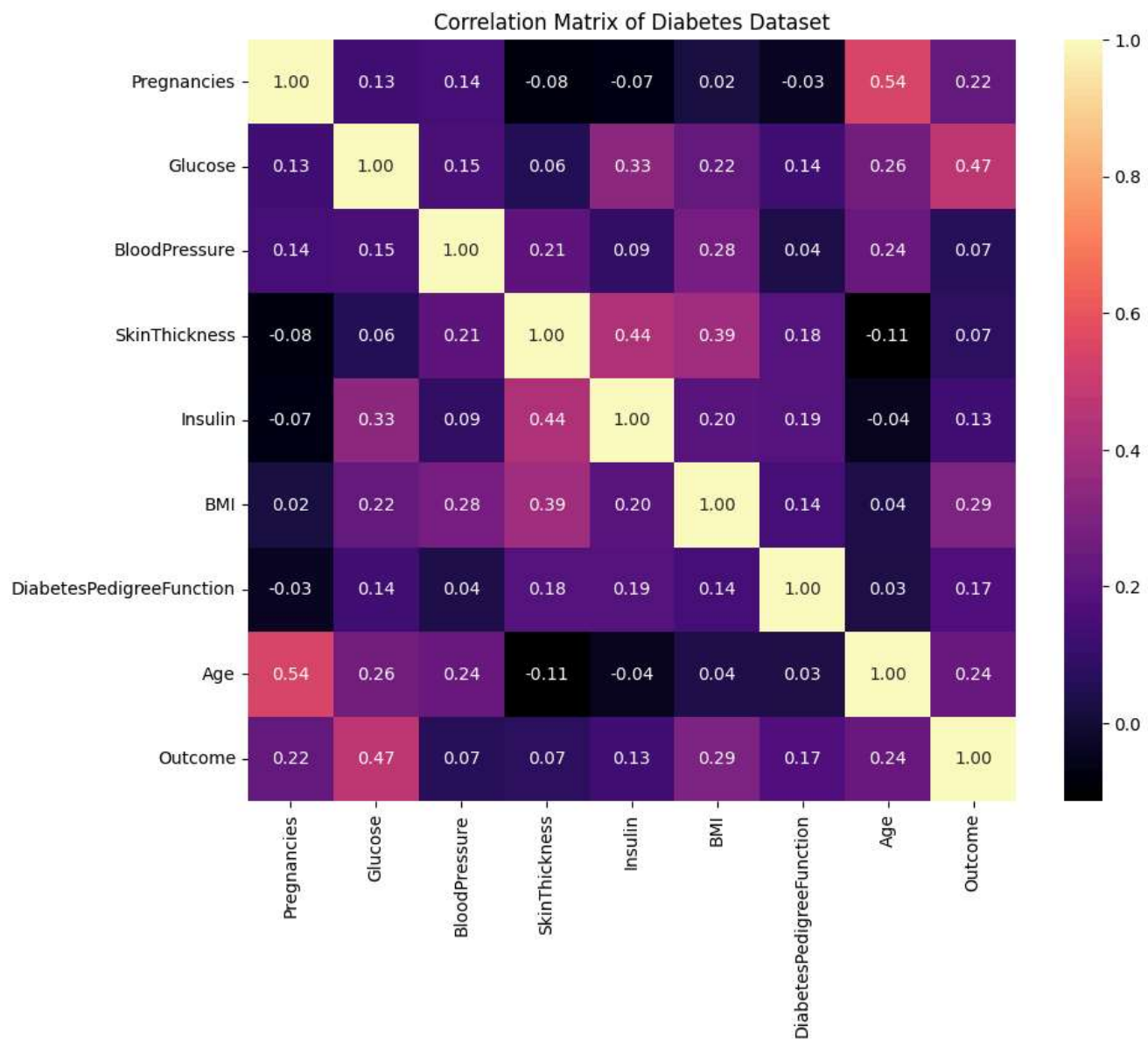
Outcome	count
0	500
1	268

dtype: int64

```
sns.countplot(x='Outcome',data=df)
plt.title("Diabetes Distribution")
plt.show()
```

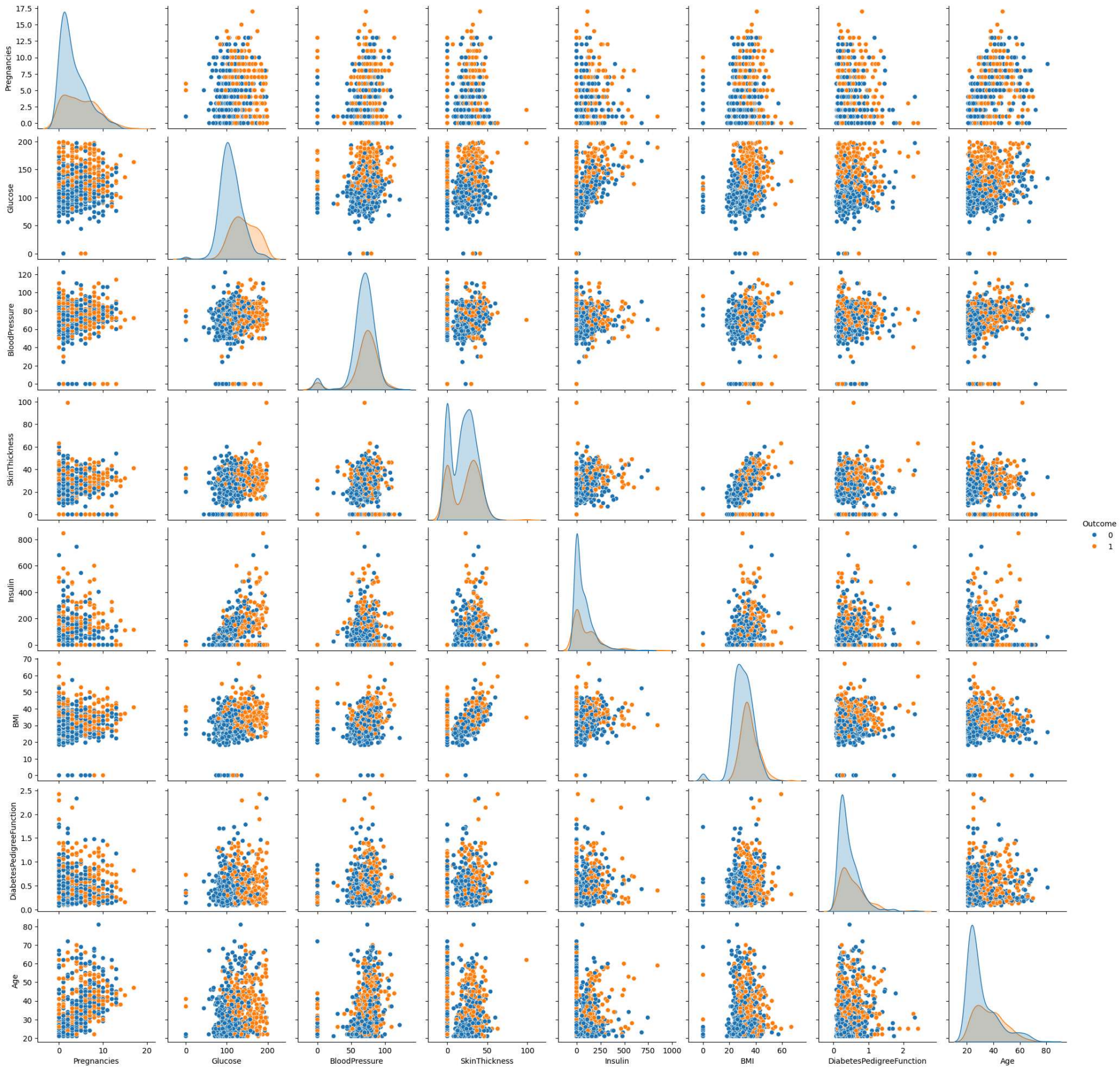



```
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, cmap='magma', fmt='.2f')
plt.title('Correlation Matrix of Diabetes Dataset')
plt.show()
```

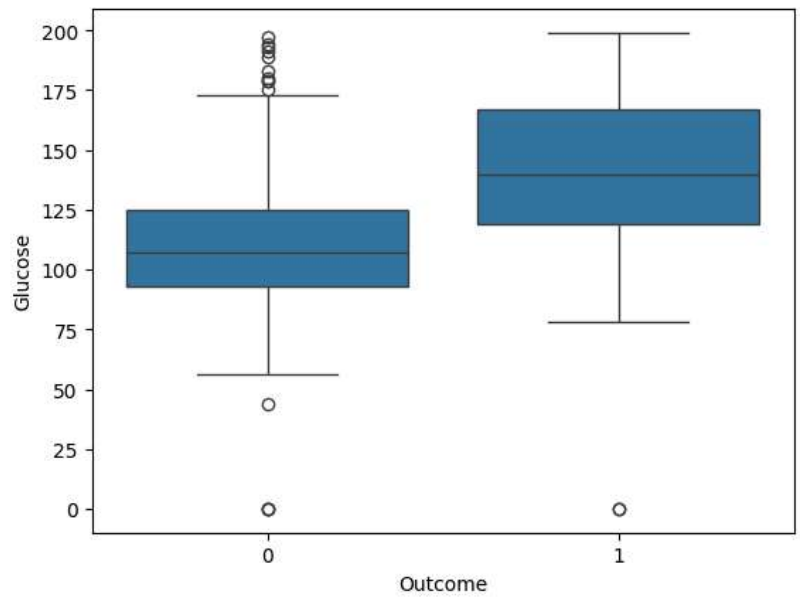



```
plt.figure(figsize=(10, 8))
sns.pairplot(df, hue='Outcome')
plt.show()
```

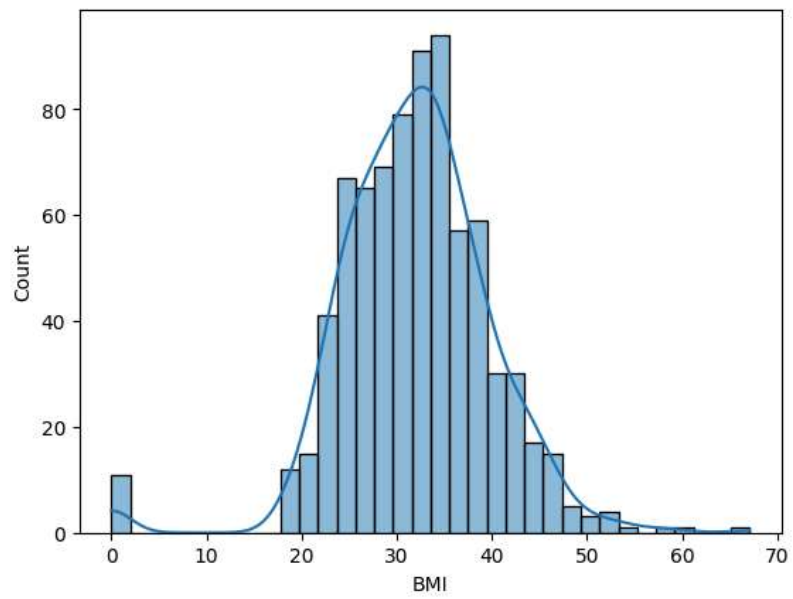
```
<Figure size 1000x800 with 0 Axes>
```



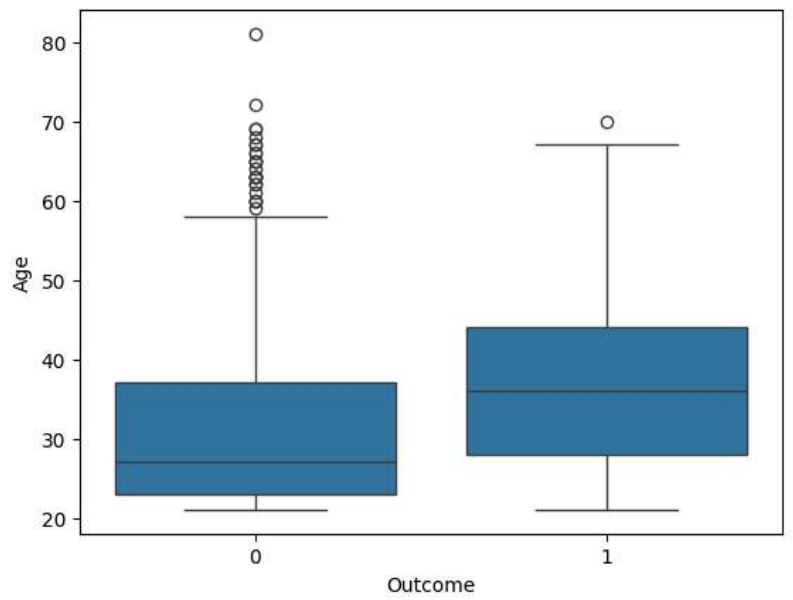

```
sns.boxplot(x='Outcome', y='Glucose',data=df)
plt.show()
```




```
sns.histplot(df['BMI'],kde=True)
plt.show()
```




```
sns.boxplot(x='Outcome', y='Age',data=df)
plt.show()
```




```
df['Glucose_BMI_Interaction'] = df['Glucose'] * df['BMI']
df['Glucose_Age_Interaction'] = df['Glucose'] * df['Age']
df['BMI_Age_Interaction'] = df['BMI'] * df['Age']
df['Pregnancies_Age_Interaction'] = df['Pregnancies'] * df['Age']


df['Glucose_Insulin_Interaction'] = df['Glucose'] * df['Insulin']
df['BMI_Insulin_Interaction'] = df['BMI'] * df['Insulin']

df['Glucose_BMI_Ratio'] = df['Glucose'] / (df['BMI'] + 1)
df['Insulin_Glucose_Ratio'] = df['Insulin'] / (df['Glucose'] + 1)
df['SkinThickness_BMI_Ratio'] = df['SkinThickness'] / (df['BMI'] + 1)
df['Glucose_Age_Ratio'] = df['Glucose'] / (df['Age'] + 1)
```

```
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 - Q1

df=df[~((df < (Q1-1.5*IQR)) | (df > (Q3+1.5*IQR))).any(axis=1)]
```

```
df
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome	Glucose_BMI_Interaction	Glucose_Age_Interaction	BMI_Age_Interact
0	6	148	72	35	0	33.6	0.627	50	1	4972.8	7400	168
1	1	85	66	29	0	26.6	0.351	31	0	2261.0	2635	82
3	1	89	66	23	94	28.1	0.167	21	0	2500.9	1869	59
5	5	116	74	0	0	25.6	0.201	30	0	2969.6	3480	76
6	3	78	50	32	88	31.0	0.248	26	1	2418.0	2028	80
...	...	...	...	...	...	...	...	...	...	...	...	
762	9	89	62	0	0	22.5	0.142	33	0	2002.5	2937	74
764	2	122	70	27	0	36.8	0.340	27	0	4489.6	3294	99
765	5	121	72	23	112	26.2	0.245	30	0	3170.2	3630	78
766	1	126	60	0	0	30.1	0.349	47	1	3792.6	5922	141
767	1	93	70	31	0	30.4	0.315	23	0	2827.2	2139	69

584 rows × 19 columns

Next steps: Generate code with df New interactive sheet

```
X=df.drop('Outcome',axis=1)
y=df['Outcome']
```

```
X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.2,random_state=42
    )
```

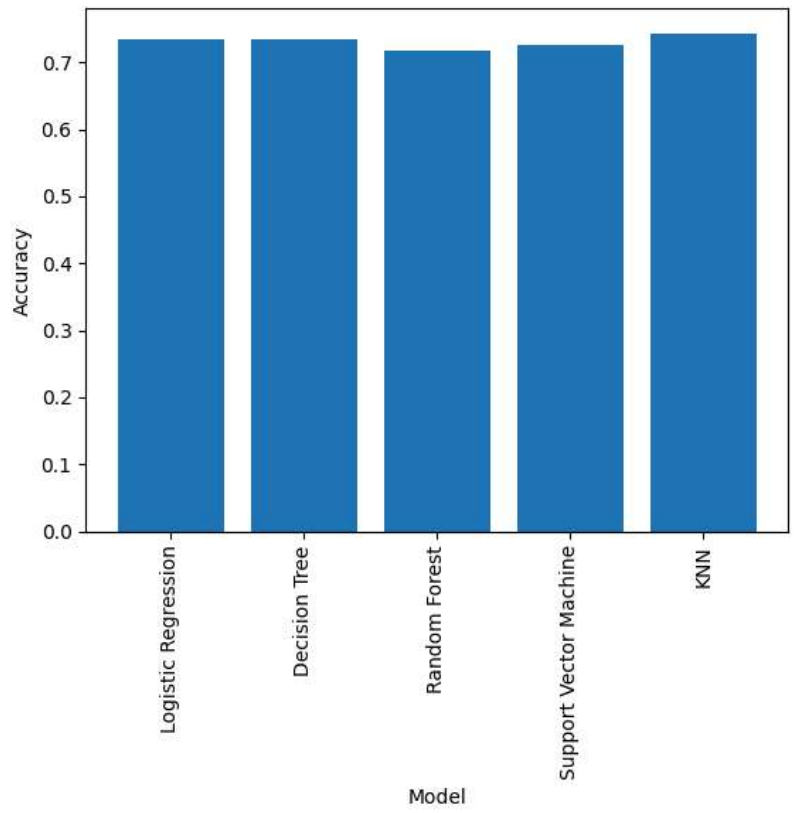
```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(
    X,y,test_size=0.2,random_state=42,stratify=y
    )
```

```
from imblearn.over_sampling import SMOTE
sm=SMOTE(random_state=42)
X_train,y_train=sm.fit_resample(X_train, y_train)
```

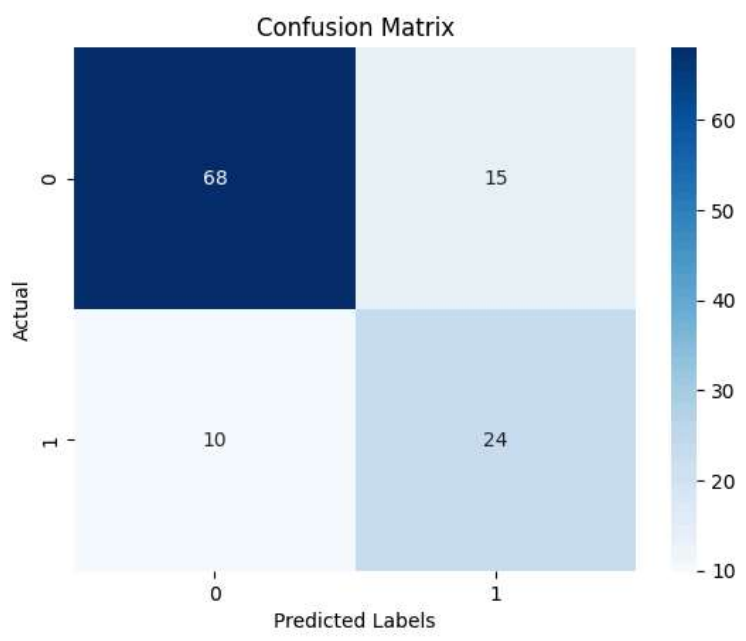
```
from sklearn.metrics import accuracy_score, precision_score,recall_score, f1_score
import pandas as pd

models= {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "Support Vector Machine": SVC(),
    "KNN": KNeighborsClassifier()
}
```

```
# Create Dataframe
results_df = pd.DataFrame(results)
# Display results
print(results_df)


plt.bar(results_df['Model'], results_df['Accuracy'])
plt.xlabel('Model')
plt.ylabel('Accuracy')
plt.xticks(rotation=90)
plt.show()
```

```
                    Model  Accuracy  Precision    Recall  \
0     Logistic Regression  0.735043   0.534884  0.676471
1           Decision Tree  0.735043   0.538462  0.617647
2           Random Forest  0.717949   0.512195  0.617647
3  Support Vector Machine  0.726496   0.521739  0.705882
4                     KNN  0.743590   0.545455  0.705882

                                F1 Score
0  <function f1_score at 0x7a3e42d2be20>
1  <function f1_score at 0x7a3e42d2be20>
2  <function f1_score at 0x7a3e42d2be20>
3  <function f1_score at 0x7a3e42d2be20>
4  <function f1_score at 0x7a3e42d2be20>
```




```
cm=confusion_matrix(y_test,pred_rf)
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("Actual")
plt.show()
```



```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Ensemble Stacking

```
from sklearn.ensemble import StackingClassifier
stacking_model = StackingClassifier(
    estimators=[
        ('dt', DecisionTreeClassifier()),
        ('rf', RandomForestClassifier()),
        ('svm', SVC())
    ],
    final_estimator=LogisticRegression()
)
stacking_model.fit(X_train, y_train)
```

```
StackingClassifier
dt
DecisionTreeClassifier
rf
RandomForestClassifier
svm
SVC
final_estimator
LogisticRegression
```

```
probs=stacking_model.predict_proba(X_test)[:,1]
def get_confidence(prob):
  if prob > 0.8:
    return"High Confidence"
  elif prob > 0.6:
    return "Medium Confidence"
  else:
    return "Low Confidence"
confidence = [get_confidence(p) for p in probs]
```

```
len(probs)
```

```
117
```

```
confidence
```

```
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Medium Confidence',
 'Low Confidence',
 'Low Confidence',
 'High Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'High Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'High Confidence',
 'High Confidence',
 'Low Confidence',
 'High Confidence',
 'Low Confidence',
 'High Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'High Confidence',
 'Low Confidence',
 'Medium Confidence',
 'Medium Confidence',
 'Low Confidence',
 'Low Confidence',
 'Medium Confidence',
 'High Confidence',
 'High Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'Low Confidence',
 'High Confidence',
 'Low Confidence',
 'Low Confidence',
 'Medium Confidence',
 'Medium Confidence',
 'Low Confidence',
 'Low Confidence',
 'Medium Confidence',
 'High Confidence',
 'Low Confidence',
 'Low Confidence']
```

```
#setting risk level
probs=stacking_model.predict_proba(X_test)[:,1]
def risk_level(prob):
  if prob < 0.3:
    return"Low risk"
  elif prob < 0.7:
    return "Medium risk"
  else:
```

```
        return "High risk"
    risk = [risk_level(p) for p in probs]
```

```
risk
```

```
 'Low risk',
 'Low risk',
 'Low risk',
 'Low risk',
 'Low risk',
 'Low risk',
 'Low risk',
 'Medium risk',
 'Low risk',
 'Low risk',
 'Medium risk',
 'Medium risk',
 'Low risk',
 'High risk',
 'Low risk',
 'Low risk',
 'Medium risk',
 'Low risk',
 'Low risk',
 'High risk',
 'Low risk',
 'Low risk',
 'Low risk',
 'Medium risk',
 'Medium risk',
 'High risk',
 'High risk',
 'Low risk',
 'High risk',
 'Low risk',
 'Low risk',
 'Low risk',
 'Medium risk',
 'Low risk',
 'High risk',
 'Low risk',
 'Medium risk',
 'High risk',
 'Low risk',
 'Medium risk',
 'High risk',
 'High risk',
 'Low risk',
 'Low risk',
 'Low risk',
 'Low risk',
 'High risk',
 'Medium risk',
 'Low risk',
 'Medium risk',
 'High risk',
 'Low risk',
 'Low risk',
 'Medium risk',
 'High risk',
 'Medium risk',
 'Low risk']
```

```
models["stacking"]=stacking_model
```

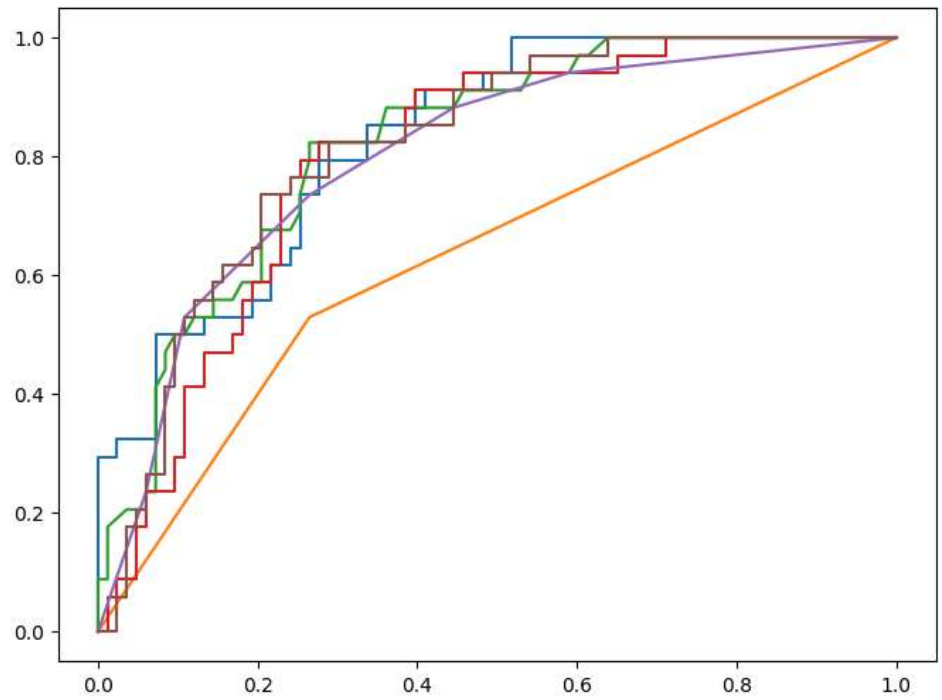
```
models
```

```
{'Logistic Regression': LogisticRegression(),
 'Decision Tree': DecisionTreeClassifier(),
 'Random Forest': RandomForestClassifier(),
 'Support Vector Machine': SVC(),
 'KNN': KNeighborsClassifier(),
 'stacking': StackingClassifier(estimators=[('dt', DecisionTreeClassifier()),
                                ('rf', RandomForestClassifier()),
                                ('svm', SVC())],
                    final_estimator=LogisticRegression())}
```

```
from sklearn.metrics import roc_auc_score
y_prob=stacking_model.predict_proba(X_test)[:,1]
roc_auc=roc_auc_score(y_test,y_prob)
print("ROC AUC Score:",roc_auc)
```

```
ROC AUC Score: 0.8182140326009921
```

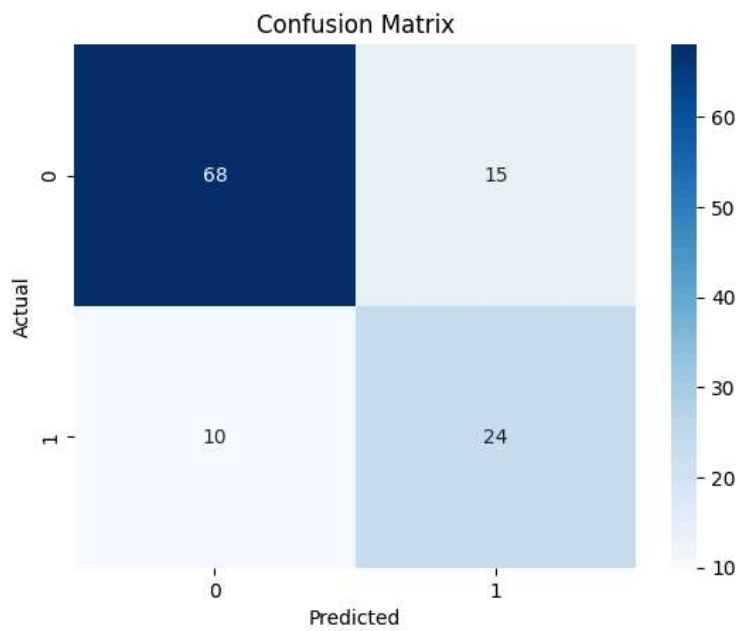
```
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
for name, model in models.items():
    #train model
    model.fit(X_train, y_train)

    # Get probability scores
    if hasattr(model, 'predict_proba'):
        y_score = model.predict_proba(X_test)[:, 1]
    else:
        y_score = model.decision_function(X_test)

    # ROC calculation (OUTSIDE if-else)
    fpr, tpr, _ = roc_curve(y_test, y_score)
    roc_auc = auc(fpr, tpr)

    # Plot each model
    plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.2f})')
```




```
y_pred_binary=(y_prob>0.5).astype(int)
print(classification_report(y_test, y_pred_binary))
```

```
              precision    recall  f1-score   support

           0       0.84      0.82      0.83        83
           1       0.58      0.62      0.60        34

    accuracy                           0.76       117
   macro avg       0.71      0.72      0.71       117
weighted avg       0.77      0.76      0.76       117
```

```
cm=confusion_matrix(y_test,pred_rf)
sns.heatmap(cm,annot=True,fmt='d',cmap='Blues')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Voting

```
from sklearn.ensemble import VotingClassifier
vote =VotingClassifier(
    estimators=[
        ('rf', RandomForestClassifier()),
        ('lr', LogisticRegression()),
        ('svm', SVC(probability=True))],
    voting='soft'
)
vote.fit(X_train, y_train)
```

```
VotingClassifier
rf: RandomForestClassifier
lr: LogisticRegression
svm: SVC
```

```
models
```

```
{'Logistic Regression': LogisticRegression(),
 'Decision Tree': DecisionTreeClassifier(),
 'Random Forest': RandomForestClassifier(),
 'Support Vector Machine': SVC(),
 'KNN': KNeighborsClassifier(),
 'stacking': StackingClassifier(estimators=[('dt', DecisionTreeClassifier()),
                                ('rf', RandomForestClassifier()),
                                ('svm', SVC())],
                    final_estimator=LogisticRegression())}
```

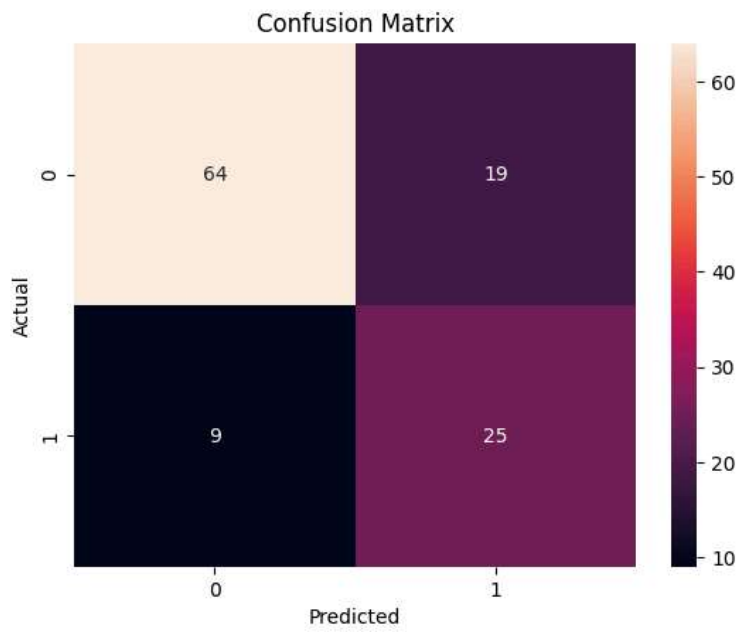
```
y_prob=vote.predict_proba(X_test)[:,1]
roc_auc=roc_auc_score(y_test,y_prob)
print("ROC AUC Score:",roc_auc)
```

```
ROC AUC Score: 0.8256555634301913
```

```
y_pred_binary=(y_prob>0.5).astype(int)
print(classification_report(y_test, y_pred_binary))
```

```
              precision    recall  f1-score   support

           0       0.88      0.77      0.82        83
           1       0.57      0.74      0.64        34

    accuracy                           0.76       117
   macro avg       0.72      0.75      0.73       117
weighted avg       0.79      0.76      0.77       117
```

```
cm=confusion_matrix(y_test,y_pred_binary)
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



Hybrid Voting and ensemble

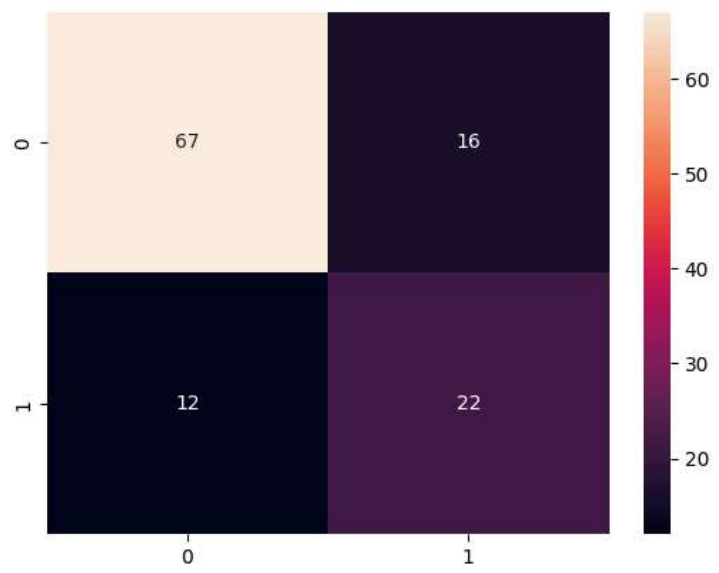
```
p1 = stacking_model.predict_proba(X_test)[:,1]
p2 = vote.predict_proba(X_test)[:,1]
rf_model = models["Random Forest"]
p3 = rf_model.predict_proba(X_test)[:,1]


final_prob = (0.4 * p1) + (0.3 * p2) + (0.3 * p3)
final_pred = (final_prob > 0.5).astype(int)
```

```
print(classification_report(y_test, final_pred))
```

```
              precision    recall  f1-score   support

           0       0.85      0.81      0.83        83
           1       0.58      0.65      0.61        34

    accuracy                           0.76       117
   macro avg       0.71      0.73      0.72       117
weighted avg       0.77      0.76      0.76       117
```

```
cm = confusion_matrix(y_test, final_pred)
sns.heatmap(cm, annot=True, fmt='d')
```

```
<Axes: >
```



XGBoost

```
!pip install xgboost
```

```
Requirement already satisfied: xgboost in /usr/local/lib/python3.12/dist-packages (3.2.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.0.2)
Requirement already satisfied: nvidia-nccl-cu12 in /usr/local/lib/python3.12/dist-packages (from xgboost) (2.29.7)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from xgboost) (1.16.3)
```

```
Q1 = df.quantile(0.25)
Q3 = df.quantile(0.75)
IQR = Q3 -Q1
print(IQR)
```

```
Pregnancies                       5.000000
Glucose                          33.000000
BloodPressure                    16.000000
SkinThickness                    32.000000
Insulin                         106.500000
BMI                               8.425000
DiabetesPedigreeFunction          0.338500
Age                              15.000000
Outcome                           1.000000
Glucose_BMI_Interaction        1558.725000
Glucose_Age_Interaction        2159.500000
BMI_Age_Interaction             560.475000
Pregnancies_Age_Interaction     191.000000
Glucose_Insulin_Interaction   12454.500000
BMI_Insulin_Interaction        3330.700000
Glucose_BMI_Ratio                 1.386634
Insulin_Glucose_Ratio             0.926152
SkinThickness_BMI_Ratio           0.938922
Glucose_Age_Ratio                 1.525422
dtype: float64
```

```
from xgboost import XGBClassifier
xgb=XGBClassifier(
    n_estimators=350,
    learning_rate=0.05,
    max_depth=8,
    subsample=0.85,
    colsample_bytree=0.85,
    random_state=42
)

xgb.fit(X_train,y_train)
y_pred=xgb.predict(X_test)
y_prob=xgb.predict_proba(X_test)[:,1]
```

```
threshold=0.50
y_pred=(y_prob > threshold).astype(int)
```

```
from sklearn.model_selection import cross_val_score
scores= cross_val_score(xgb,X, y, cv=5, scoring='roc_auc')
print("Mean_Auc:",scores.mean())
```

```
Mean_Auc: 0.7954866064717234
```

```
from sklearn.metrics import roc_auc_score, accuracy_score
print("Accuracy:", accuracy_score(y_test,y_pred))
print("AUC:", roc_auc_score(y_test,y_prob))
```

```
Accuracy: 0.7521367521367521
AUC: 0.7771084337349398
```

```
cm=confusion_matrix(y_test,y_pred)
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```